Introduction to Program Synthesis (SS 2026)

Chapter 4 - Advanced Methodologies

Dr. rer. nat. Roman Kalkreuth

Chair for AI Methodology (**AIM**), Faculty of Computer Science,
RWTH Aachen University, Germany

Advanced Methodologies
Neural Program Synthesis: Recurrent Neural Networks

- Feed-forward Neural Networks $\rightarrow$ uni-directional dataflow
  - $\rightsquigarrow$ Each input is treated independently

  Recurrent Neural Networks (RNN) $\rightarrow$ processing of sequential data via feedback loops
  - $\rightsquigarrow$ Sequence modelling of time series, speech, text, **code**, music, ...
  - $\rightsquigarrow$ Loop-like architecture $\rightarrow$ self-looping or *recurrent workflow*
  - $\rightsquigarrow$ Memoisation of past input states $\rightarrow$ remembering and using previous inputs in the hidden layer

Advanced Methodologies

Neural Program Synthesis: Recurrent Neural Networks

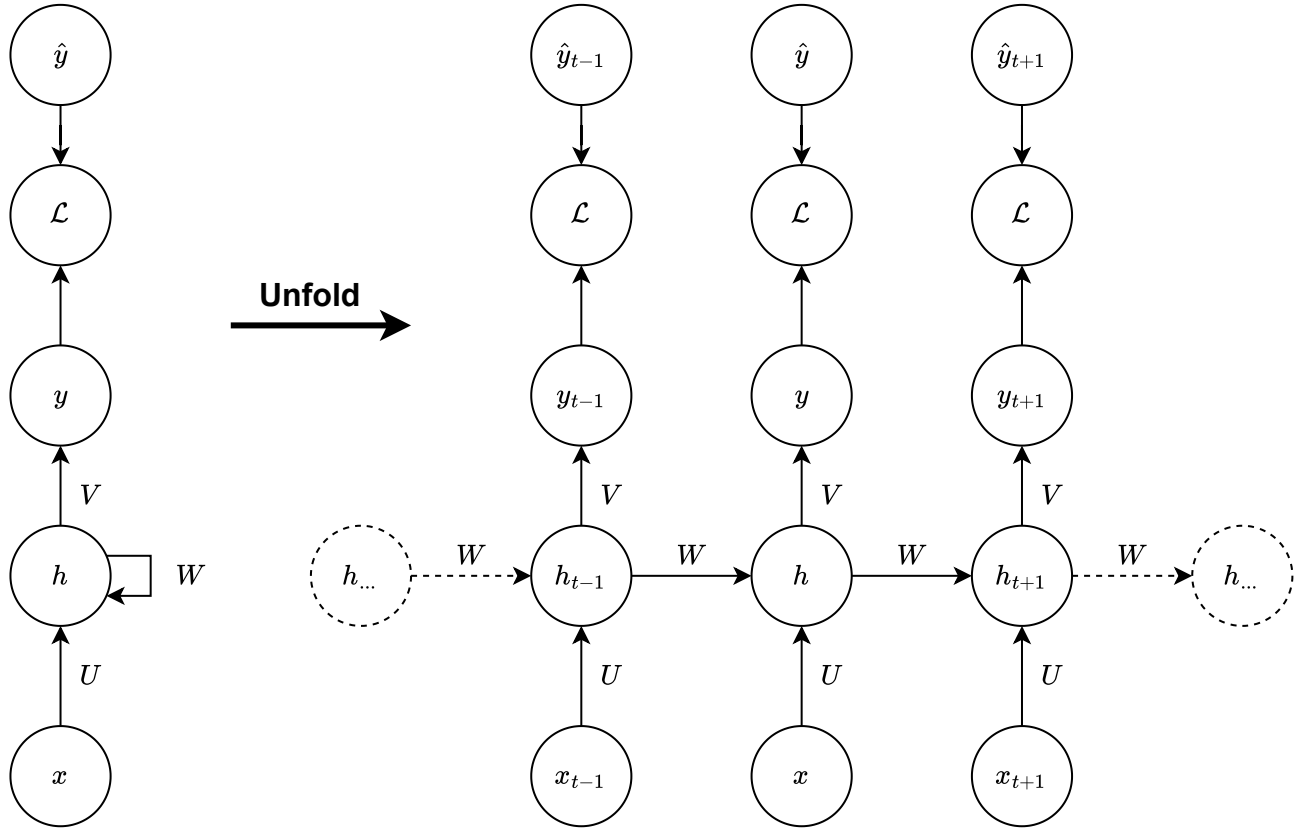

Figure: Recurrent Neural Network (RNN)

Advanced Methodologies
Neural Program Synthesis: Recurrent Neural Networks

- A RNN can be considered a function $f(x_t, h_t, \theta) \mapsto (y_t, h_{t+1})$
  - $\rightsquigarrow$ x_t : input vector
  - $\rightsquigarrow$ h_t : hidden vector
  - $\rightsquigarrow$ y_t : output vector
  - $\rightsquigarrow$ θ : hyperparameters
- The input vector x_t is mapped into an output y_t
  - $\rightsquigarrow$ hidden vector h_t serves as an *memory*
- *Transformation* of an input to an output at each step t
  - $\rightsquigarrow$ U, V and $W \rightarrow$ weight matrices
  - $\rightsquigarrow$ b and $c \rightarrow$ bias vectors
  - $\rightsquigarrow$ Step-wise update $\rightarrow$ back-propagation through time (BPTT)

Advanced Methodologies
Neural Program Synthesis: Recurrent Neural Networks

- Update equations are applied from $t = 1$ to $t = \tau$
  - $\rightsquigarrow$ $a^t = b + Wh^{t-1} + Ux^t$
  - $\rightsquigarrow$ $h^t = tanh(a^t)$
  - $\rightsquigarrow$ $y^t = c + Vh^t$

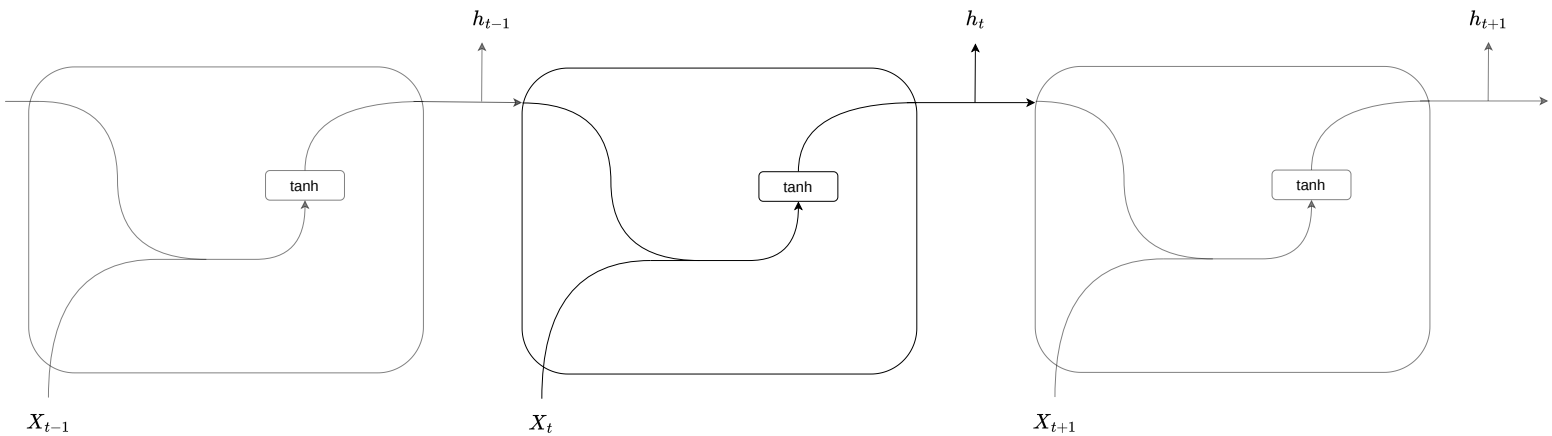

Figure: Repeating module in a standard RNN

Advanced Methodologies

Neural Program Synthesis: Long-short term memory (LSTM)

- RNNs are prone to vanishing or exploding gradients
  - $\rightsquigarrow$ Long-term gradients that are back-propagated through time can *vanish*
  - $\rightsquigarrow$ **Gated RNN (GRU)** $\rightarrow$ gating mechanism used in recurrent neural networks
  - $\rightsquigarrow$ Optimise recurrent neural networks through efficient memory mechanisms
- **Long-short term memory (LSTM)** $\rightarrow$ learning when to remember and when to forget information
  - $\rightsquigarrow$ Ability to decide when inputs should be remembered or ignored in the hidden state

Advanced Methodologies

Neural Program Synthesis: Long-short term memory (LSTM)

- LSTM architecture $\rightarrow$ gated memory cell
  - $\rightsquigarrow$ **Input gate** $I_t \rightarrow$ decides when information is added to the cell
  - $\rightsquigarrow$ **Forget gate** $F_t \rightarrow$ resets the content of the cell
  - $\rightsquigarrow$ **Output gate** $O_t \rightarrow$ determines the output of the cell
- Hidden state unit $\rightarrow$ replaced by a LSTM cell

Advanced Methodologies

Neural Program Synthesis: Long-short term memory (LSTM)

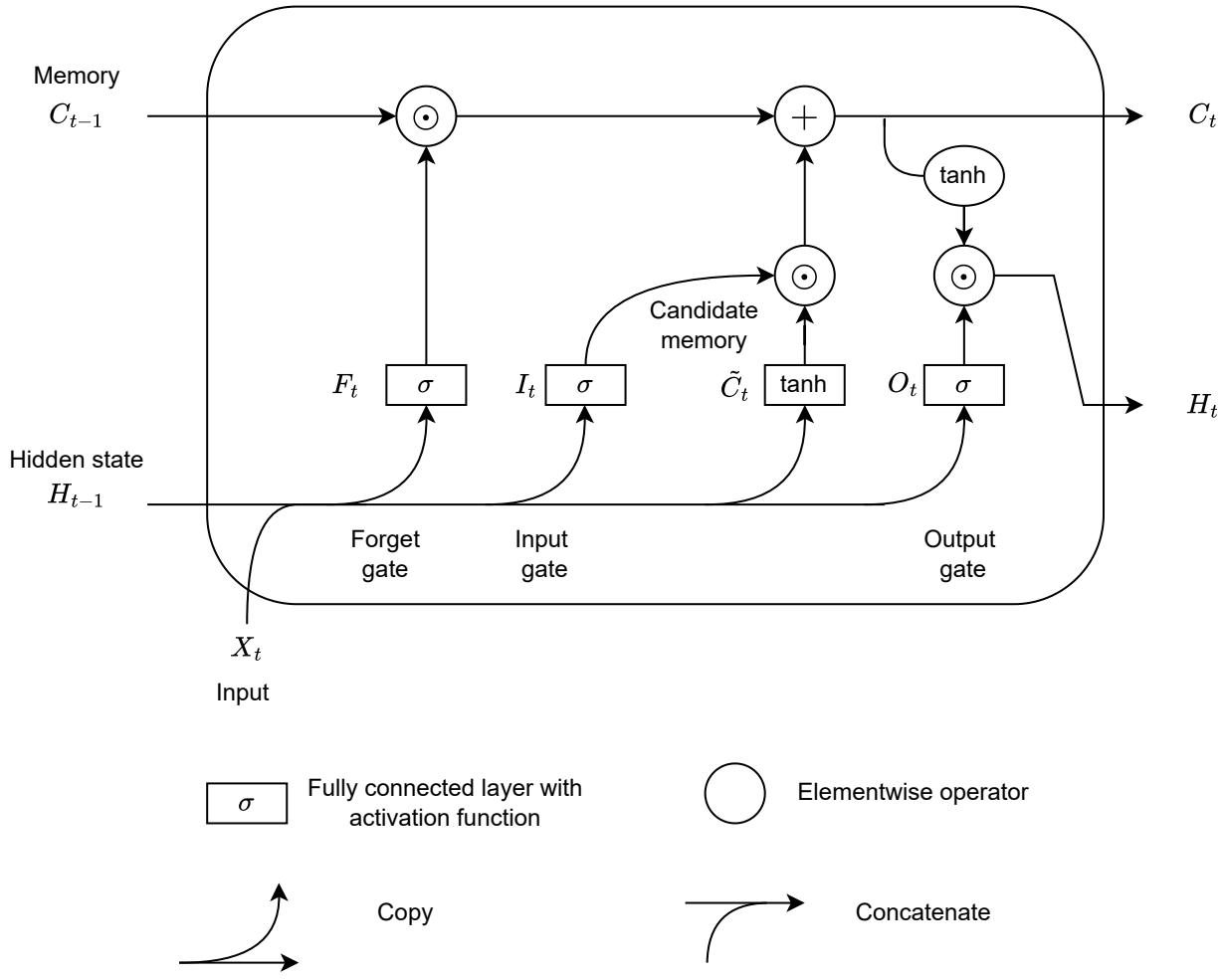

Figure: Long-short term memory (LSTM)

Advanced Methodologies
Neural Program Synthesis: Long-short term memory (LSTM)

- LSTM variables (d and h refer to the number of features and hidden units)
  - $\rightsquigarrow$ $x_t \in \mathbb{R}^d$: input vector to the LSTM cell
  - $\rightsquigarrow$ $f_t \in (0,1)^h$: activation vector of the forget gate
  - $\rightsquigarrow$ $i_t \in (0,1)^h$(: activation vector of the input or update gate
  - $\rightsquigarrow$ $o_t \in (0,1)^h$: activation vector of the output gate
  - $\rightsquigarrow$ $h_t \in (-1,1)^h$: hidden state vector or output vector of the LSTM cell
  - $\rightsquigarrow$ $\tilde{c}_t \in (-1,1)^h$: activation vector of the cell input
  - $\rightsquigarrow$ $c_t \in \mathbb{R}^h$: cell state vector
  - $\rightsquigarrow$ $W \in \mathbb{R}^{h \times d}$, $U \in \mathbb{R}^{h \times h}$ and $b \in \mathbb{R}^h$: weight matrices and bias vector parameters
- Activation functions
  - $\rightsquigarrow$ σ: sigmoid function
  - $\rightsquigarrow$ tanh: hyperbolic tangent function.

Advanced Methodologies

Neural Program Synthesis: Long-short term memory (LSTM)

- Forward pass of a LSTM cell
  - $\rightsquigarrow f_t = \sigma(x_t W_f + h_{t-1} U_f + b_f)$
  - $\rightsquigarrow i_t = \sigma(x_t W_i + h_{t-1} U_i + b_i)$
  - $\rightsquigarrow o_t = \sigma(x_t W_o + h_{t-1} U_o + b_o)$
  - $\rightsquigarrow \tilde{c}_t = \tanh(x_t W_c + h_{t-1} U_c + b_c)$
  - $\rightsquigarrow c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}$
  - $\rightsquigarrow h_t = o_t \odot \tanh(c_t)$

Advanced Methodologies
Neural Program Synthesis: Long-short term memory (LSTM)

- Candidate memory cell $\tilde{c}_t \in \mathbb{R}^{n \times h}$
  - $\rightsquigarrow$ $\tanh(x_t W_{xc} + h_{t-1} W hc + b_c)$
- Memory cell c_t
  - $\rightsquigarrow$ $c_t = F_t \odot c_{t-1} + i_t \odot \tilde{c}_t$
- Hiden state h_t
  - $\rightsquigarrow$ $h_t = o_t \odot \tanh(C_t)$
  - $\rightsquigarrow$ Values of h_t are then in the interval $(-1, 1)$

Advanced Methodologies

Neural Program Synthesis: Long-short term memory (LSTM)

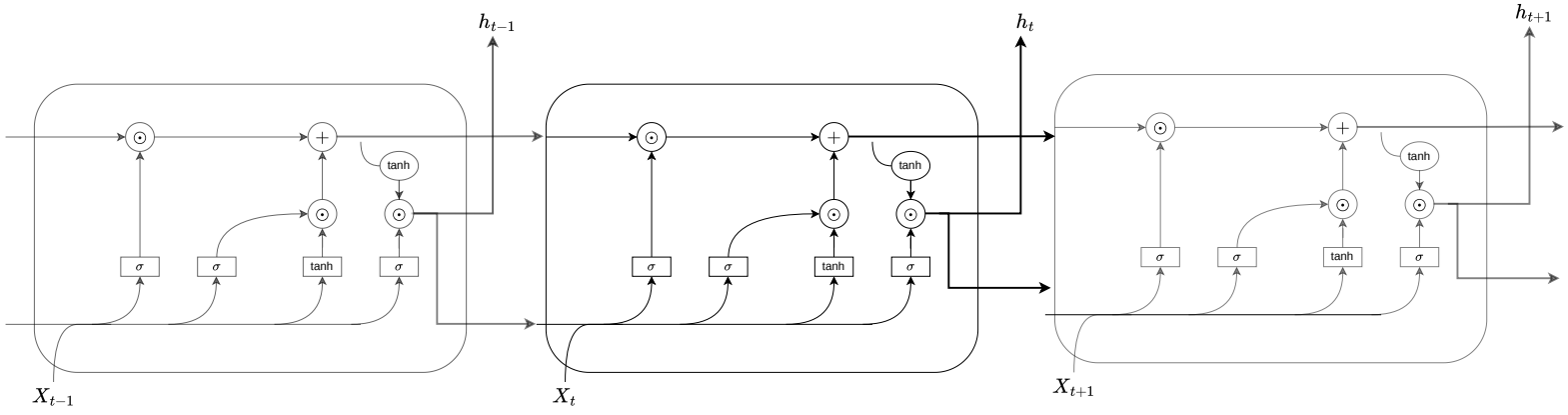

Figure: Chain of LSTM cells

Advanced Methodologies

Neural Program Synthesis: Autoencoder

- ANN that is trained to attempt to copy the input to the corresponding outputs
  - $\rightsquigarrow$ Mapping of an input x to an output x' (also called reconstruction)
  - $\rightsquigarrow$ Encodes a latent representation or code h
- A hidden layer h represents the code
  - $\rightsquigarrow$ Latent space reprensetation or encoding of the input
- Mainly consists of two components
  - $\rightsquigarrow$ Encoder function $f = f(x) \rightarrow$ mapping from x to h
  - $\rightsquigarrow$ Decoder function for reconstruction $g = g(h) \rightarrow$ mapping to h to r

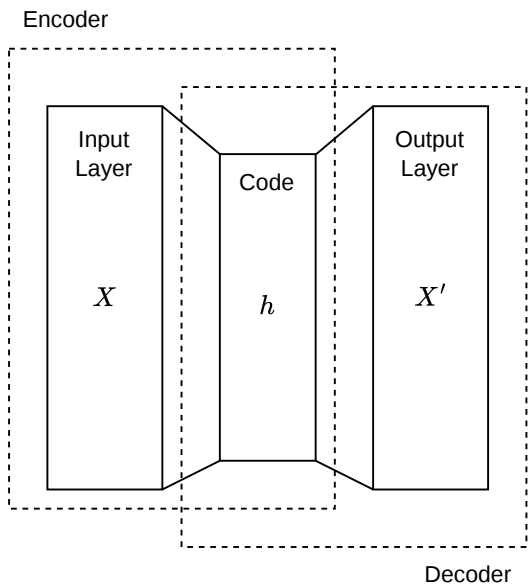

Figure: Architecture of an autoencoder

Advanced Methodologies
Neural Program Synthesis: Transformer

- Successor of RNN's and LSTM for automated natural language processing (NLP)
  - $\rightsquigarrow$ Extension via attention mechanisms
  - $\rightsquigarrow$ Transformers neglect recurrent structures $\rightarrow$ focus on attention mechanism
  - $\rightsquigarrow$ Imitation of human cognitive attention
- Transformer calculate a weighting for each word in the context of the embedding
  - $\rightsquigarrow$ Embedding is based on encoder-decoder architecture
  - $\rightsquigarrow$ Embedding layer weights are adjusted during training
  - $\rightsquigarrow$ Transformation via **Word2Vec**

Advanced Methodologies
Neural Program Synthesis: Transformer

- **Embedding** $\rightarrow$ preparation of the inputs for the transformer model
  - $\rightsquigarrow$ Breaks raw text into smaller pieces (called tokens)
  - $\rightsquigarrow$ Converted into numerical vectors called embeddings
- Word2Vec $\rightarrow$ vector representations of words
  - $\rightsquigarrow$ Words in similar contexts $\rightarrow$ mapping to vectors
  - $\rightsquigarrow$ Distance is measured with **cosine similarity**
  - $\rightsquigarrow$ *Text-to-token* $\rightarrow$ **tokenizing**
- **Positional encoding** $\rightarrow$ sequential order of the words is respected
- **Attention mechanism** $\rightarrow$ weighting tokens based on their importance
  - $\rightsquigarrow$ Self attention $\rightarrow$ capturing of long-range dependencies without sequential processing
  - $\rightsquigarrow$ Multi-headed attention $\rightarrow$ enabling focus on various aspects of the input data simultaneously

Advanced Methodologies
Neural Program Synthesis: Transformer

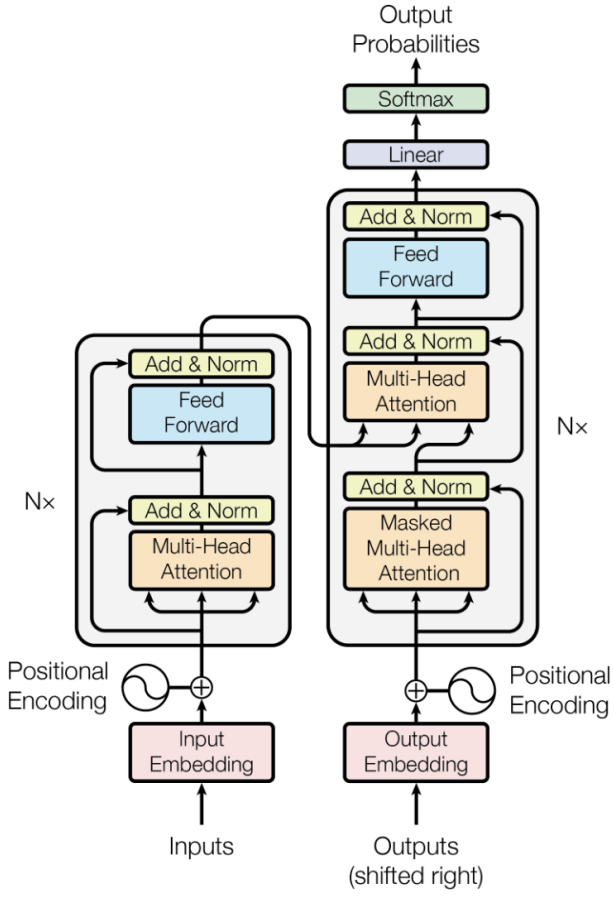

Figure: Transformer (Source: Vaswani et al. (2017))